

มหาวิทยาลัยมหิดล
Mahidol University



ITCS215
Introduction to Interactive Multimedia Technology

Semester 1/2026

Introduction to Image Processing/Computer Vision

Assoc. Prof. Dr. Worapan Kusakunniran

TensorFlow / Keras

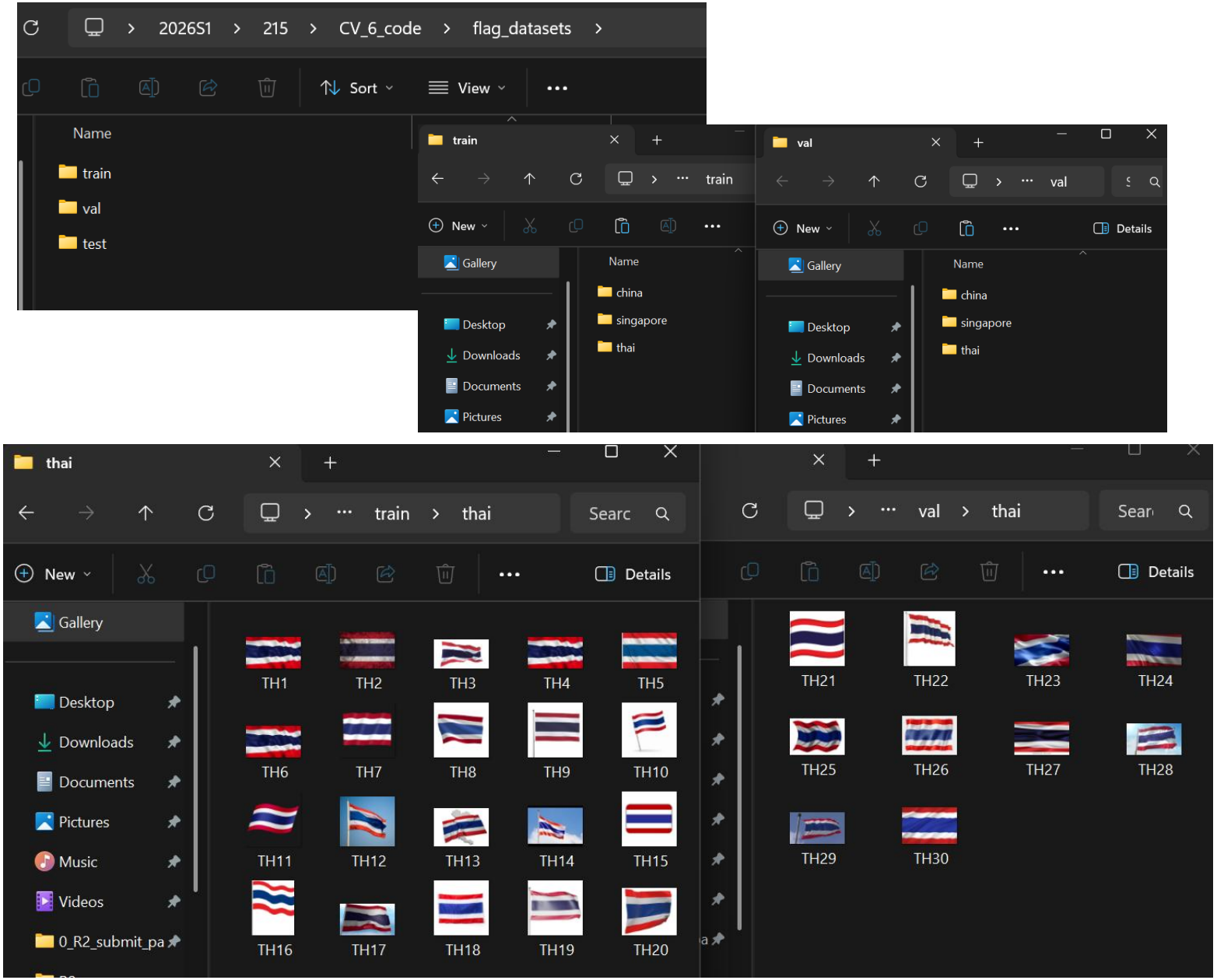
- Keras
  - Keras is an open-source **neural-network library** written in **Python**.
  - It can run on top of **TensorFlow**, Microsoft Cognitive Toolkit, R, Theano, or PlaidML.
- TensorFlow
  - TensorFlow is a free and open-source software **library for dataflow and differentiable programming across a range of tasks**.

wikipedia.org

Dataset

- Define your task with at least 3 classes e.g.,
  - Classifying 3 types of national flags
    - Thai
    - Singapore
    - China
- Collect images from open-source or your own images
  - 30 images of each class (at least)
- Prepare training and validating folders
  - 20 images for training
  - 10 images for validating
- Prepare testing images
  - At least one unseen image of each class
- A better solution
  - Yolo detection (object detection) in years 3 and 4

Dataset


2026S1 › 215 › CV_6_code › flag_datasets
Sort
View
Name
train
val
test
train
New
Gallery
Desktop
Downloads
Documents
Pictures
Name
china
singapore
thai
val
Details
thai
Searc
Music
Videos
0_R2_submit_pa
TH1
TH2
TH3
TH4
TH5
TH6
TH7
TH8
TH9
TH10
TH11
TH12
TH13
TH14
TH15
TH16
TH17
TH18
TH19
TH20
TH21
TH22
TH23
TH24
TH25
TH26
TH27
TH28
TH29
TH30

Define the simple net

Load-in data samples

```
import keras
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten
from keras.layers import Conv2D, MaxPooling2D
# Create a data generator
from keras.preprocessing.image import ImageDataGenerator

img_rows, img_cols = 100, 100
input_shape = (img_rows, img_cols, 3)
num_classes=20
train_datagen = ImageDataGenerator(rescale=1./255)
train_generator = train_datagen.flow_from_directory('/content/drive/MyDrive/dataset/Georgia
TechFaceDatabase/train/',target_size=(img_rows,img_cols),batch_size=10,class_mode='categori
cal', shuffle=True)
step_size_train=train_generator.n/train_generator.batch_size

val_datagen = ImageDataGenerator(rescale=1./255)
val_generator = val_datagen.flow_from_directory('/content/drive/MyDrive/dataset/GeorgiaTech
FaceDatabase/val/',target_size=(img_rows,img_cols),batch_size=10,class_mode='categorical',
shuffle=True)
step_size_val=val_generator.n/val_generator.batch_size
```

```
Found 200 images belonging to 20 classes.
Found 100 images belonging to 20 classes.
```

Define CNN architecture

```
model = Sequential()
model.add(Conv2D(32, kernel_size=(3, 3),
                 activation='relu',
                 input_shape=input_shape))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))
model.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_16 (Conv2D)           (None, 98, 98, 32)        896
_________________________________________________________________
conv2d_17 (Conv2D)           (None, 96, 96, 64)        18496
_________________________________________________________________
max_pooling2d_8 (MaxPooling2 (None, 48, 48, 64)        0
_________________________________________________________________
flatten_8 (Flatten)          (None, 147456)            0
_________________________________________________________________
dense_16 (Dense)             (None, 128)               18874496
_________________________________________________________________
dense_17 (Dense)             (None, 10)                1290
=================================================================
Total params: 18,895,178
Trainable params: 18,895,178
Non-trainable params: 0
```

Train a model

```
from tensorflow.keras.optimizers import SGD, Adam

adam = Adam(lr=0.0003)
model.compile(loss=keras.losses.categorical_crossentropy,
              optimizer=adam,
              metrics=['accuracy'])
history = model.fit_generator(generator=train_generator,
                  steps_per_epoch=step_size_train,epochs=10,
                  validation_data=val_generator,
                  validation_steps=step_size_val)
```

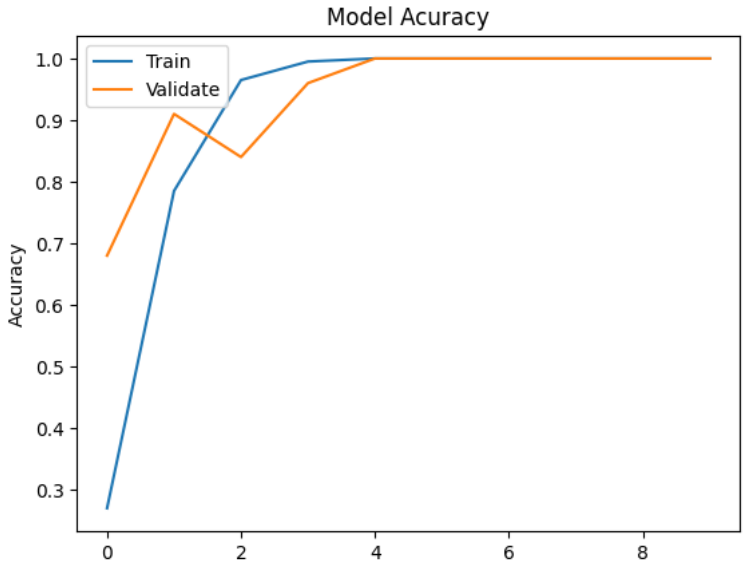
```
Epoch 1/10
20/20 [==============================] - 19s 908ms/step - loss: 4.3470 - accuracy: 0.2700 - val_loss: 1.9551 - val_accuracy: 0.6800
Epoch 2/10
20/20 [==============================] - 17s 842ms/step - loss: 1.0266 - accuracy: 0.7850 - val_loss: 0.3321 - val_accuracy: 0.9100
Epoch 3/10
20/20 [==============================] - 18s 893ms/step - loss: 0.1585 - accuracy: 0.9650 - val_loss: 0.2963 - val_accuracy: 0.8400
Epoch 4/10
20/20 [==============================] - 17s 834ms/step - loss: 0.0377 - accuracy: 0.9950 - val_loss: 0.1009 - val_accuracy: 0.9600
Epoch 5/10
20/20 [==============================] - 17s 829ms/step - loss: 0.0069 - accuracy: 1.0000 - val_loss: 0.0073 - val_accuracy: 1.0000
Epoch 6/10
20/20 [==============================] - 17s 832ms/step - loss: 0.0015 - accuracy: 1.0000 - val_loss: 0.0045 - val_accuracy: 1.0000
Epoch 7/10
20/20 [==============================] - 17s 842ms/step - loss: 7.6336e-04 - accuracy: 1.0000 - val_loss: 0.0035 - val_accuracy: 1.0000
Epoch 8/10
20/20 [==============================] - 17s 845ms/step - loss: 4.6506e-04 - accuracy: 1.0000 - val_loss: 0.0025 - val_accuracy: 1.0000
Epoch 9/10
20/20 [==============================] - 17s 843ms/step - loss: 3.5270e-04 - accuracy: 1.0000 - val_loss: 0.0020 - val_accuracy: 1.0000
Epoch 10/10
20/20 [==============================] - 18s 898ms/step - loss: 2.9935e-04 - accuracy: 1.0000 - val_loss: 0.0020 - val_accuracy: 1.0000
```

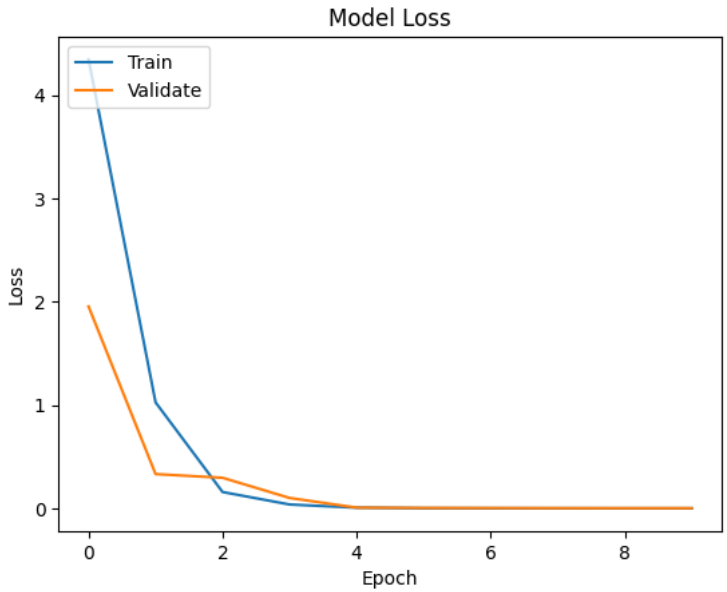
Need a face localization

Plot training and validating graphs

```
import matplotlib
import matplotlib.pyplot as plt
# Plot training accuracy values
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Acuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train','Validate'], loc='upper left')
plt.savefig('Accuracy.png')
plt.show()

# Plot training loss values
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train','Validate'], loc='upper left')
plt.savefig('Loss.png')
plt.show()
```

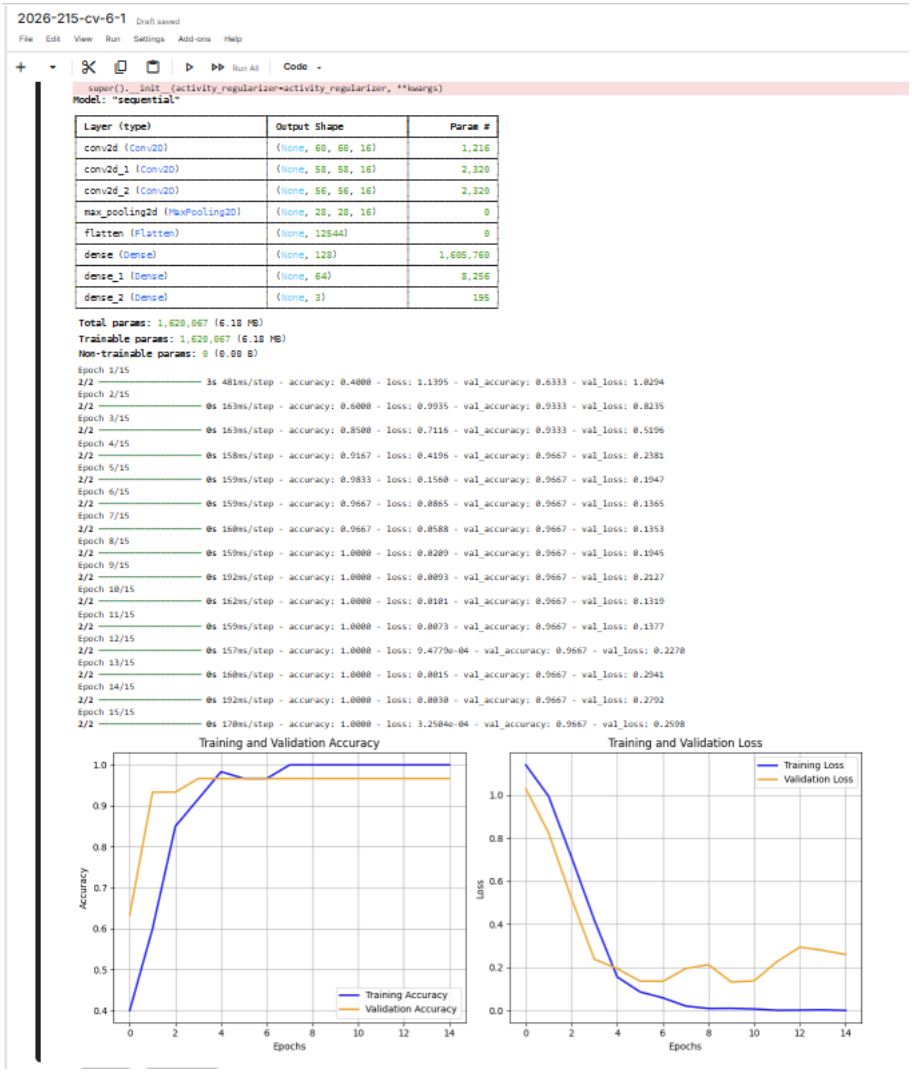




Class act 6.1

Write a python code to classify the 3 selected class with the following CNN architecture, train, and show training and validating graphs

- Input
- Conv., 16 filters, 5x5 kernel size, relu
- Conv., 16 filters, 3x3 kernel size, relu
- Conv., 16 filters, 3x3 kernel size, relu
- Maxpool, pool size = 2x2
- Flatten
- Dense, 128 nodes
- Dense, 64 nodes
- Output



Class act 6.2:
Save and load a model for predicting ID of each image

- Save model
- Load model
- Use the loaded model to predict a class of a test image

[2]:
```python
# Save the trained model
model_path = 'cnn_3class_model.keras'
model.save(model_path)

print(f"Model successfully saved to '{model_path}'")
```

```
Model successfully saved to 'cnn_3class_model.keras'
```

[3]:
```python
import tensorflow as tf

# Load the saved model
loaded_model = tf.keras.models.load_model('cnn_3class_model.keras')

print("Model successfully loaded!")
```

```python
# ==========================================================================
# Example Execution
# ==========================================================================

# List of class names matching the order in your dataset directory
class_names = ['China', 'Singapore', 'Thailand']

# Path to your target test image
test_image_path = '/kaggle/input/datasets/worapan/cv-6-1/flag_datasets/test/TH31.jpg'

# Run prediction using the loaded model
predicted_class, confidence = predict_single_image(
    image_path=test_image_path,
    model=loaded_model,
    class_names=class_names
)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step

==============================
Image Path     : /kaggle/input/datasets/worapan/cv-6-1/flag_datasets/test/TH31.jpg
Predicted Class: Thailand
Confidence     : 100.00%
==============================
```

```python
import numpy as np
import tensorflow as tf

def predict_single_image(image_path, model, class_names, img_height=64, img_width=64):
    """
    Loads, preprocesses, and predicts the class of a test image using the trained model.
    """
    # 1. Load image and resize to target input size
    img = tf.keras.utils.load_img(
        image_path,
        target_size=(img_height, img_width)
    )

    # 2. Convert image to numpy array
    img_array = tf.keras.utils.img_to_array(img)

    # 3. Normalize pixel values from [0, 255] to [0, 1]
    img_array = img_array / 255.0

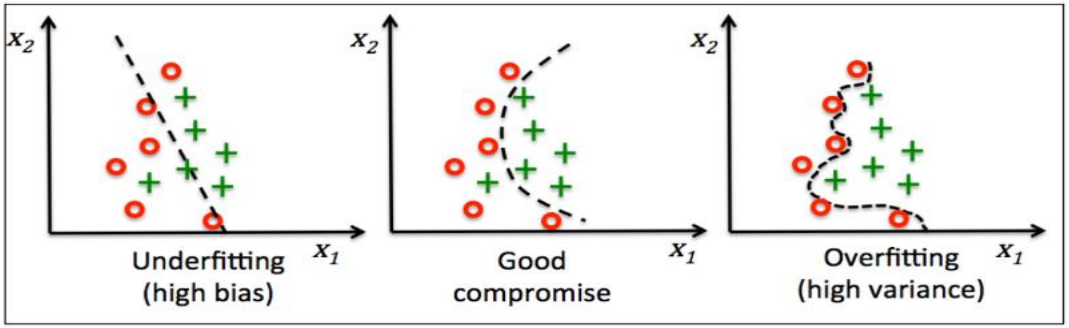
    # 4. Expand dimensions to add batch dimension -> Shape: (1, 64, 64, 3)
    img_array = tf.expand_dims(img_array, axis=0)

    # 5. Perform prediction
    predictions = model.predict(img_array)

    # 6. Extract predicted class index and confidence percentage
    predicted_class_idx = np.argmax(predictions[0])
    confidence = predictions[0][predicted_class_idx] * 100
    predicted_class_name = class_names[predicted_class_idx]

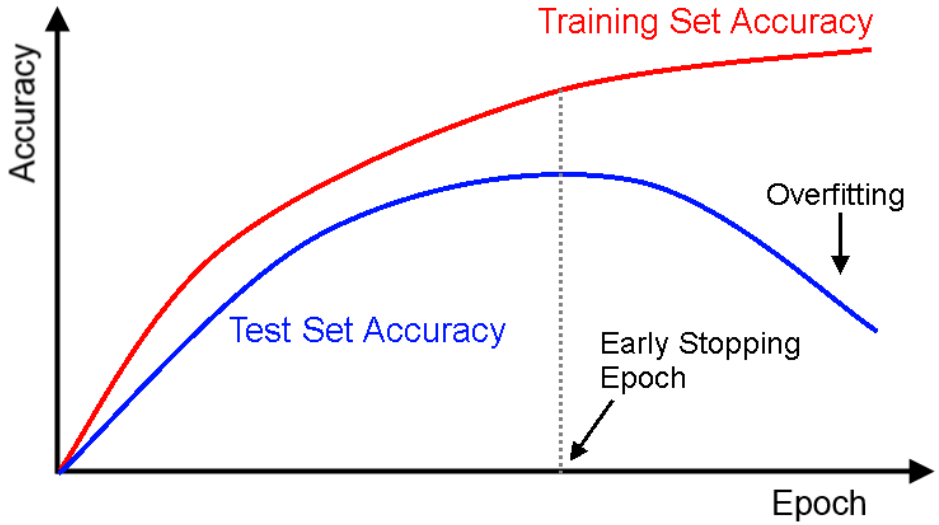
    print("\n" + "="*30)
    print(f"Image Path     : {image_path}")
    print(f"Predicted Class: {predicted_class_name}")
    print(f"Confidence     : {confidence:.2f}%")
    print("="*30)
```

Overfitting



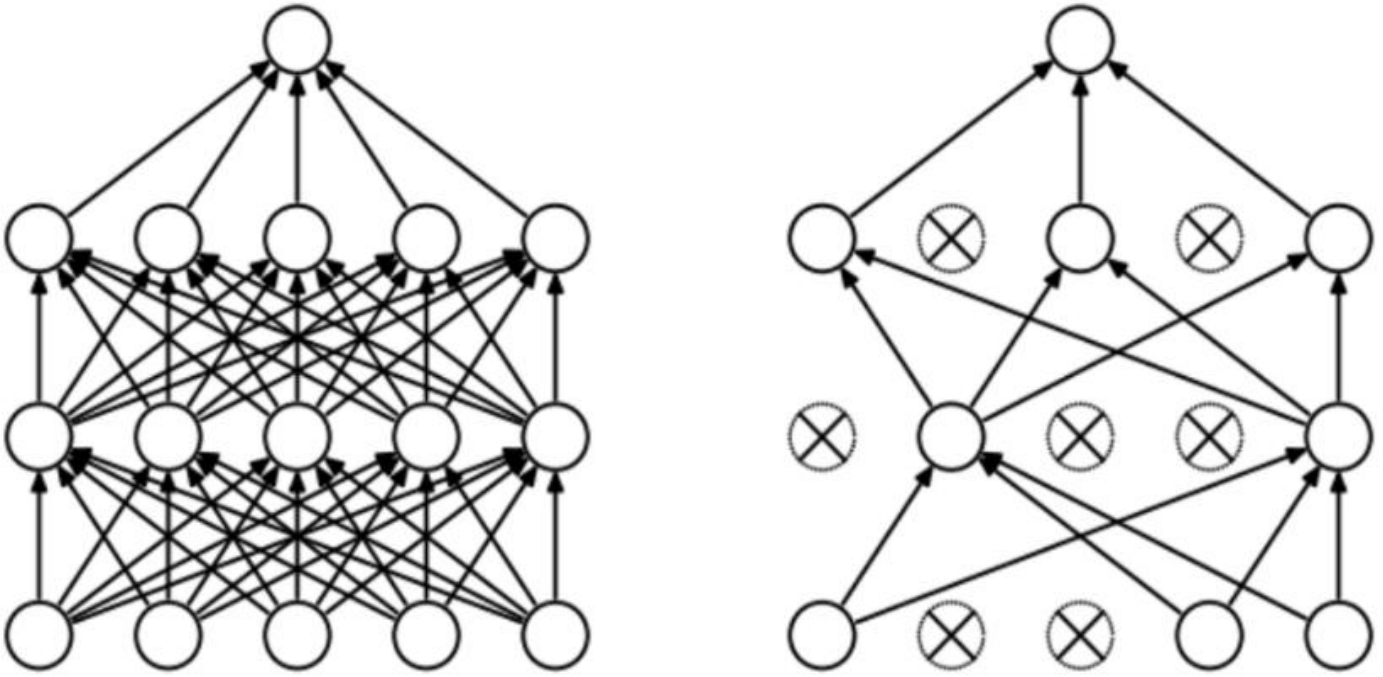
Overfitting

- Not enough data
- Too complete network
- Too many training loops



- Early stopping
- Drop out
- Data augmentation

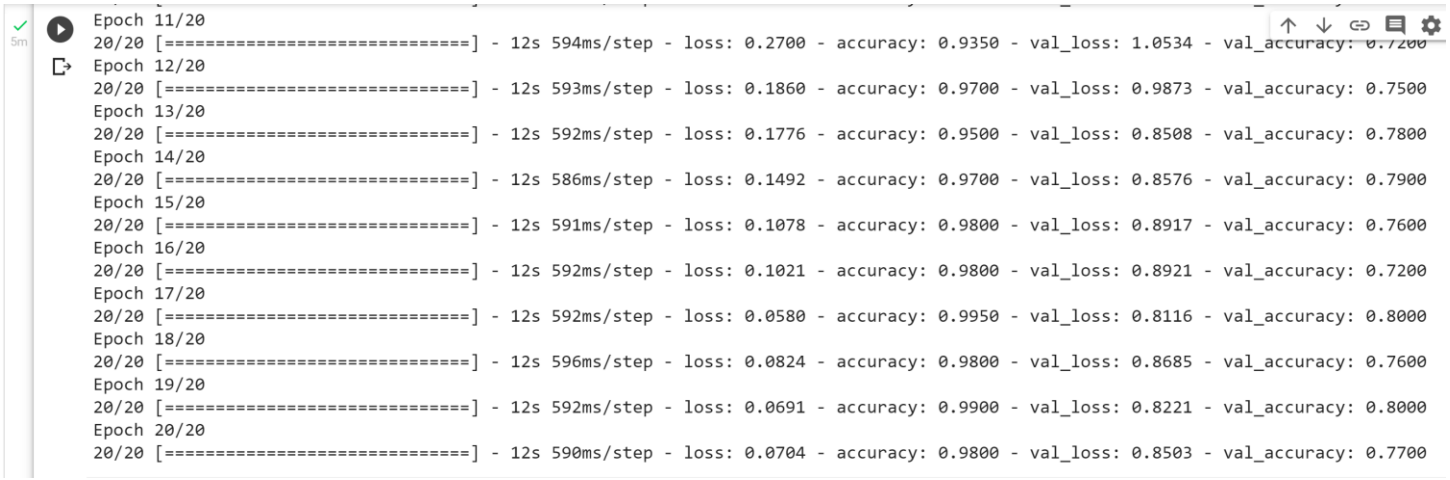
Drop out



Visualization of dropout

Drop out

```
model = Sequential()
model.add(Conv2D(32, kernel_size=(3, 3),
                 activation='relu',
                 input_shape=input_shape))
model.add(Dropout(0.2))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(Dropout(0.1))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.1))
model.add(Dense(num_classes, activation='softmax'))
model.summary()
```



Data augmentation

Why need the augmentation

- Not enough data
  - Quantity
  - Variations
- Reduce overfitting

Image Augmentation: Type



- Zoom
- Translate
- Color manipulation
- Brightness manipulation
- Blur manipulation
- Contrast manipulation
- Shear
- Flip (horizontal and vertical)
- Perspective transform
- Cropping
- Padding
- Sharpen





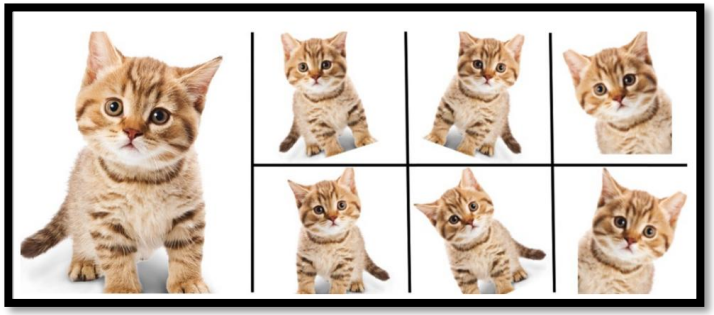




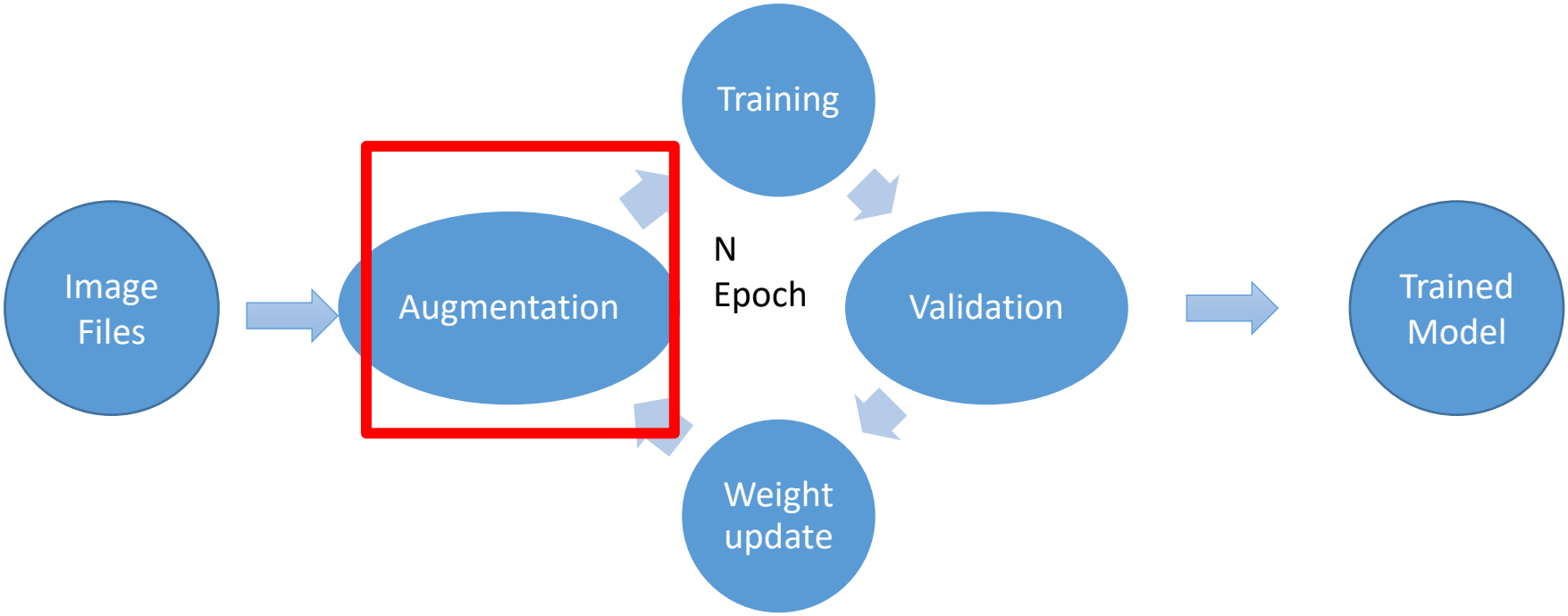
Image Augmentation: Technique

- Real-time augmentation
- Manual augmentation

Real-time augmentation

- Built in Keras's API: **ImageDataGenerator** class
  - *"Generate batches of tensor image data with real-time data augmentation. The data will be looped over (in batches)"*
- ImageDataGenerator object will generate augmented images from the input image files in every epoch at the beginning

Real-time augmentation in CNN training flow chart



Real-time augmentation

- ImageDataGenerator object's parameters
  - rotation_range *Degree range for random rotations*

```
from keras.preprocessing import image
import numpy as np
from keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
img_rows, img_cols = 100, 100
img = image.load_img('/content/drive/My Drive/faces94/9326871/9326871.1.jpg', target_size=(img_rows, img_cols))
datagen = ImageDataGenerator(rotation_range=40)
generate_plot_pics(datagen,img)
```

[-40,40]

https://keras.io/preprocessing/image/

Real-time augmentation

- ImageDataGene
  - rotation_rang




```
from keras.preproc
import numpy as np
from keras.preproc
import matplotlib.p
img_rows, img_cols
img = image.load_i
_size=(img_rows, ir
datagen = ImageDa
generate_plot_pics
```

871.1.jpg', target

[-40,40]

https://keras.io/preprocessing/image/

Real-time augmentation

- ImageDataGenerator object's parameters
  - zoom_range *Range for random zoom*

Float or [lower, upper]. Range for random zoom. If a float, [lower, upper] = [1-zoom_range, 1+zoom_range].

```
datagen = ImageDataGenerator(zoom_range=0.3)
```

[1-0.3,1+0.3] = [0.7,1.3]

https://keras.io/preprocessing/image/

Real-time augmentation

- ImageDataGenerator object's parameters
  - zoom_ran

Float or [lower

oom_range, 1+zoom_range].

datag

nge=0.3)

[1-0.3,1



https://keras.io/preprocessing/image/

Real-time augmentation

- ImageDataGenerator object's parameters
  - brightness_range *Range for picking a brightness shift value from*

```
datagen = ImageDataGenerator
(brightness_range=[0.2,1.8])
```

- produces images similar to as taken with different lighting conditions
- pass the min and the max range based on which the image will be darkened or brightened
- Values <1 darkens the image, >1 brightens the image and =1 means no change



https://theailearner.com/2019/07/06/data-augmentation-with-keras-imagedatagenerator/
https://keras.io/preprocessing/image/

Real-time augmentation

- ImageDataGenerator object's parameters
  - shear_range — *Shear Intensity (Shear angle in counter-clockwise direction in degrees)*

```
datagen =
ImageDataGenerator(shear_range=30)
```

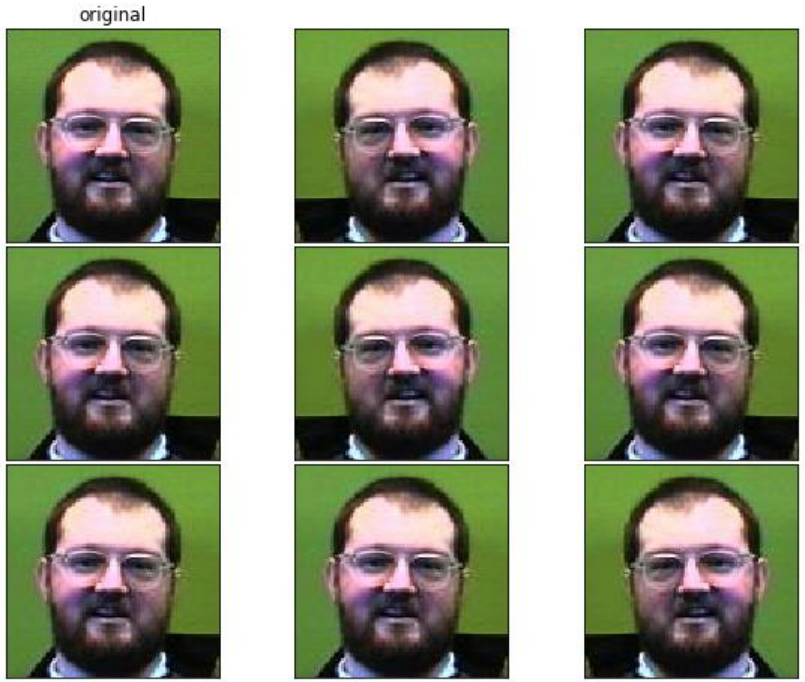


https://keras.io/preprocessing/image/

Real-time augmentation

- ImageDataGenerator object's parameters
  - horizontal_flip *Randomly flip inputs horizontally*

```
datagen =
ImageDataGenerator(horizontal_flip=True)
```

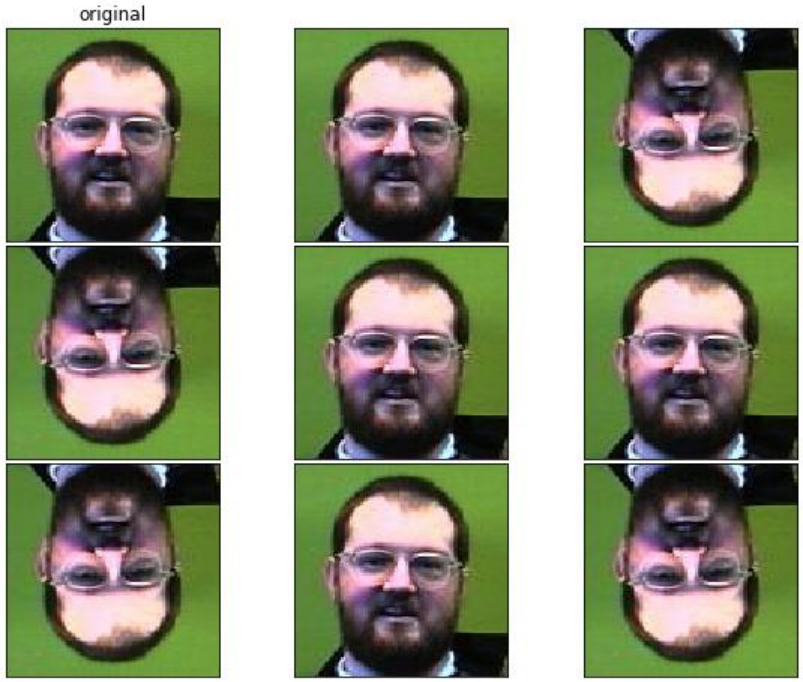


https://keras.io/preprocessing/image/

Real-time augmentation

- ImageDataGenerator object's parameters
  - vertical_flip *Randomly flip inputs vertically*

```
datagen =
ImageDataGenerator(vertical_flip =True)
```



https://keras.io/preprocessing/image/

Real-time augmentation

- ImageDataGenerator object's parameters
  - rescale *Rescaling factor*

https://keras.io/preprocessing/image/

Real-time augmentation

- After set up the ImageDataGenerator object
- Pass image files through the ImageDataGenerator object for augmentation via **flow_from_directory** function

Real-time augmentation

- Example

```
train_datagen = ImageDataGenerator(
        rescale=1./255,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True)
```

```
train_generator = train_datagen.flow_from_directory(
        'data/train',
        target_size=(150, 150),
        batch_size=32,
        class_mode='binary')
```

Manual augmentation

- Pre-augmentation images first
- Instead of pass original images through the network, we pass the already augmented images
- Use third party libraries for augment the images
  - Imgaug *https://github.com/aleju/imgaug*
  - skimage *https://scikit-image.org/*
  - OpenCV *https://opencv.org/*
  - Pillow *https://pillow.readthedocs.io/en/3.1.x/index.html*
  - **ImageDataGenerator. flow_from_directory** *set "save_to_dir" parameter*

Class act 6.3

Add

- At least 1 type of data augmentation
- 1 dropout of 0.01

```
# ==========================================================================
# 2. Data Augmentation Layers
# ==========================================================================
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),        # Augmentation Type 1: Horizontal Flip
    layers.RandomRotation(0.1),             # Augmentation Type 2: Random Rotation
    layers.RandomZoom(0.1)                  # Augmentation Type 3: Random Zoom
], name="data_augmentation")
```

```
# ==========================================================================
# 3. CNN Model Architecture
# ==========================================================================
model = models.Sequential([
    # Input Layer + Rescaling
    layers.Input(shape=(img_height, img_width, 3)),
    layers.Rescaling(1.0 / 255),

    # Integrated Data Augmentation (Active only during training)
    data_augmentation,

    # Conv Layer 1: 16 filters, 5x5 kernel, relu
    layers.Conv2D(16, (5, 5), activation='relu'),

    # Conv Layer 2: 16 filters, 3x3 kernel, relu
    layers.Conv2D(16, (3, 3), activation='relu'),

    # Conv Layer 3: 16 filters, 3x3 kernel, relu
    layers.Conv2D(16, (3, 3), activation='relu'),

    # MaxPool Layer: pool size 2x2
    layers.MaxPooling2D(pool_size=(2, 2)),

    # Flatten Layer
    layers.Flatten(),

    # Dense Layer: 128 nodes
    layers.Dense(128, activation='relu'),

    # Dropout Layer (rate = 0.01)
    layers.Dropout(0.01),

    # Dense Layer: 64 nodes
    layers.Dense(64, activation='relu'),

    # Output Layer: 3 nodes (Softmax for 3 classes)
    layers.Dense(num_classes, activation='softmax')
])

model.summary()
```